AI Finance Assistant with Risk Analysis and Deal Detection: An Integrated Architecture Combining FinGuard and DealDetective

Hema Surya B *Department of Computer Science and Engineering*
[Vit ,Chennai]
Chennai,India
hemasurya106@email.com

***Abstract*—Personal financial decision-making is increasingly challenged by information overload, impulse spending, and the difficulty of identifying genuine value in a dynamic e-commerce landscape. This paper presents an integrated AI-driven financial assistant that combines two complementary subsystems: FinGuard, a personal finance intelligence engine for risk analysis, and DealDetective, an AI-powered product deal finder and price monitoring system. FinGuard employs a staged machine-learning pipeline—encompassing K-Means clustering, Isolation Forest anomaly detection, and a Decision Outcome Learning Engine (DOLE) built on Random Forest—to analyze daily spending behavior and provide actionable risk verdicts. A Gemini 2.5 Flash-powered Text-to-SQL chat interface enables natural-language queries over personal financial data. DealDetective complements this by scraping product listings from major Indian e-commerce platforms (Amazon, Flipkart, Snapdeal), generating 768-dimensional semantic fingerprints using Google's `text-embedding-004` model stored in a `pgvector`-enabled PostgreSQL database, offering AI-generated product analysis (jargon decoding, eco-scoring), and sending automated email price alerts. Together, the two systems form a unified pipeline that assists users in both controlling outgoing expenditure and maximizing value in purchasing decisions. Experimental evaluation demonstrates the effectiveness of the full ML pipeline in classifying high-risk spending days, while qualitative assessment validates the accuracy of the deal-detection and product-matching components. The integrated system is implemented as a full-stack application comprising two FastAPI backends, a React frontend for FinGuard, and a React frontend for DealDetective, with a shared PostgreSQL database layer.**

***Index Terms*—Personal Finance, Anomaly Detection, Machine Learning, Deal Detection, Large Language Models, Gemini API, K-Means Clustering, Isolation Forest, Random Forest, pgvector, Natural Language Processing, Text-to-SQL, Price Monitoring, e-Commerce**

I. Introduction

The rapid proliferation of digital payment systems and e-commerce platforms has created a dual challenge for modern consumers: managing personal expenditure against dynamically shifting spending patterns, and discerning genuine deals from manipulated discount displays. Traditional budgeting tools are predominantly rule-based and fail to adapt to individual behavioral rhythms, while product comparison portals rely on static price snapshots without providing semantic understanding of product value or personalized affordability context.

This work presents an integrated AI Finance Assistant that addresses both dimensions. The system consists of two tightly coupled but independently deployable modules:

- **FinGuard** — A personal financial intelligence system that tracks daily expenses, classifies spending behavior using unsupervised machine learning, detects anomalous expenditure patterns, simulates future purchase affordability using a burn-rate model, learns from past decision outcomes via a reinforcement-inspired learning engine (DOLE), and supports natural-language financial queries via a Text-to-SQL interface powered by the Gemini LLM.
- **DealDetective** — A product deal intelligence system that extracts real-time product data (price, specifications, images) from Amazon.in, Flipkart, and Snapdeal via a multi-strategy web scraper, computes semantic product fingerprints using Google's embedding model for similarity-based product matching ("Twin Finder"), generates structured AI analysis of product specifications, maintains price history, and delivers automated email alerts when prices drop to user-defined targets.

The core contribution of this paper is the description and evaluation of this two-module integrated architecture. By combining outgoing spend risk analysis with incoming purchase opportunity detection, the system provides a holistic view of a user's financial decisions.

The remainder of the paper is organized as follows. Section II reviews related work. Section III presents the overall system architecture. Section IV describes the algorithms and models used. Section V provides implementation details. Section VI discusses security and ethical AI integration. Section VII presents results and analysis. Section VIII discusses findings and limitations, and Section IX concludes.

II. Related Work

A. AI-Powered Personal Finance

Personal finance management systems have evolved from simple budgeting apps to ML-driven platforms. Mint [1] and YNAB represent rule-based systems that require manual categorization. Research into automated transaction categorization has applied convolutional neural networks [2] and transformer-based models with increasing accuracy. Anomaly detection in financial transactions has been extensively studied in the

context of fraud detection [3], [20], with Isolation Forest [4] and Autoencoder-based approaches consistently achieving high AUC scores. Our work adapts Isolation Forest to the domain of personal spending pattern analysis, a less-explored application.

B. Behavioral Spending Analysis

Cluster-based segmentation of consumer spending behavior has been studied using K-Means and Gaussian Mixture Models [5]. Our staged pipeline extends this by adapting the analysis strategy to data maturity: heuristic rules for sparse data (fewer than 15 days), and full ML for mature datasets, a design choice inspired by cold-start mitigation strategies [6].

C. LLM-Driven Financial Interfaces

The application of Large Language Models to structured data retrieval via Text-to-SQL [7] has gained significant traction. Works such as DINSQL [8] and ACT-SQL [9] demonstrate LLM capability in generating syntactically and semantically correct queries. Furthermore, modern interfaces draw on adaptive conversational strategies akin to online learning in spoken dialogue systems [19]. Our implementation uses Gemini 2.5 Flash Lite with multi-query decomposition to answer complex financial questions.

D. Product Price Monitoring and Deal Detection

Web scraping for price tracking has been applied by commercial services (CamelCamelCamel [10]) and studied academically [11]. Semantic product similarity for "find similar" features has been explored using BERT embeddings [12], product knowledge graphs [13], and pre-trained embeddings for entity resolution [18]. The underlying challenge of finding twin deals shares foundations with duplicate record detection [17]. Our use of pgvector [15] for vector similarity search within PostgreSQL leverages efficient Hierarchical Navigable Small World (HNSW) algorithms [21], offering a practical, unified-database approach compared to dedicated vector databases.

E. Multi-Platform E-Commerce Data Extraction

Structured data extraction across heterogeneous e-commerce platforms, a domain extensively analyzed in information extraction surveys [16], is challenging due to anti-scraping measures and dynamic rendering. Works by [14] study platform-specific CSS selector strategies, while others address JavaScript-rendered content via headless browsers. Our scraper service uses a dual-strategy approach: static `requests`-based fetching with Playwright fallback.

III. System Architecture

A. Overall Pipeline

The integrated system operates as a two-backend, two-frontend architecture backed by a shared PostgreSQL instance extended with the `pgvector` extension. Figure 1 illustrates the high-level architecture.

B. FinGuard Architecture

FinGuard comprises four primary components: the **Expense Ingestion Layer**, the **ML Pipeline**, the **DOLE (Decision Outcome Learning Engine)**, and the **Query Interface**.

Expense Ingestion Layer exposes REST endpoints via FastAPI for manual expense entry, bill scanning (OCR via Gemini Vision), and user profile management. All data is persisted to PostgreSQL in an `expenses` table with fields: `user_id`, `date`, `category`, `amount`, `payment_mode`, and `day_type`.

ML Pipeline is triggered synchronously after every expense addition. It aggregates daily data, determines the data maturity stage, and applies the appropriate analysis model. Results are stored in a `recommendations` table.

DOLE runs as a two-phase learning loop: an outcome labeling phase evaluates decisions made more than 7 days ago, and a model training phase fits a Random Forest classifier on labeled decision records.

Query Interface processes natural-language questions by converting them to SQL via Gemini 2.5 Flash Lite, executing the queries safely, and synthesizing human-readable answers.

C. DealDetective Architecture

DealDetective comprises three primary components: the **Scraping and Extraction Layer**, the **AI Analysis Engine**, and the **Alert and Monitoring Service**.

Scraping and Extraction Layer implements a two-strategy fetch mechanism: synchronous `requests`-based HTML fetching with retry logic (3 retries with backoff), followed by a Playwright headless browser fallback for JavaScript-rendered pages. Platform-specific CSS selectors target the product title, current price, original price (MRP), and discount information for Amazon, Flipkart, and Snapdeal.

AI Analysis Engine sends the extracted HTML to Gemini 2.5 Flash Lite for structured JSON extraction of product details (title, price, specifications, eco-meter rating, jargon explanations). The engine also generates a 768-dimensional semantic embedding using Google's `text-embedding-004` model, stored as a `pgvector` column in the `products` table, enabling cosine-similarity based "Twin Finder" queries.

Alert and Monitoring Service is a background async loop (`asyncio`) that checks all active `PriceAlert` records at configurable intervals (default: 3600 seconds). When the current price falls at or below the target price, it dispatches an email alert via SMTP through the `email_service` module and deactivates the alert.

IV. Methodology

A. FinGuard: Staged ML Pipeline

The FinGuard ML pipeline implements three distinct stages based on the number of days of expense data available for a user.

1) Stage 1: New User (0–2 Days): When fewer than 3 days of data exist, no statistical model is applied. The system returns onboarding messages encouraging continued data entry. This avoids premature analysis on insufficient data.

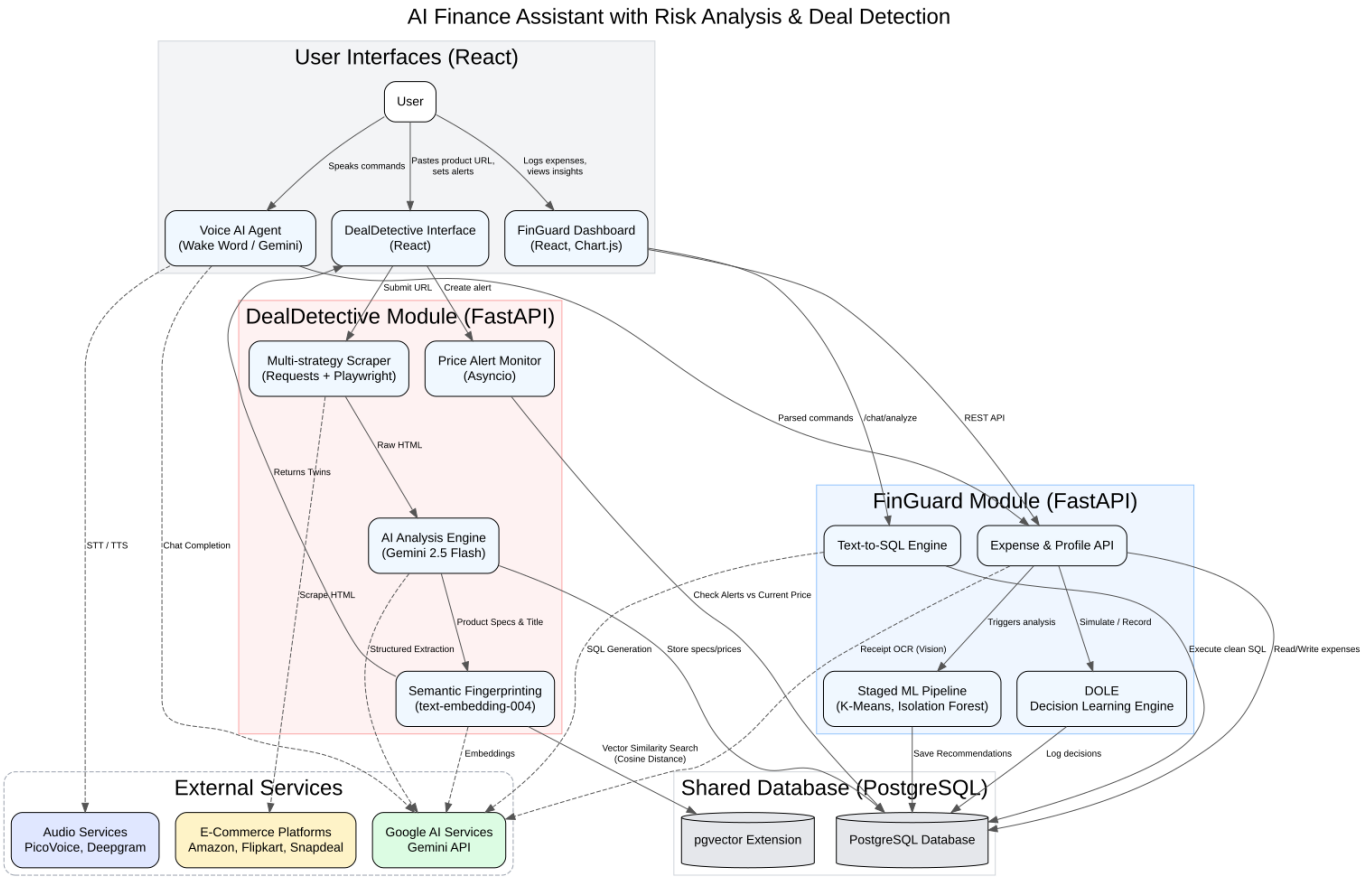

Fig. 1. High-level architecture of the AI Finance Assistant. The left branch shows the FinGuard pipeline (expense ingestion → ML pipeline → DOLE → React dashboard). The right branch shows the DealDetective pipeline (URL scraping → Gemini analysis → pgvector storage → React frontend).

2) Stage 2: Early User (3–14 Days): For users with 3 to 14 days of data, a statistical heuristic compares each day's total spend to the rolling mean:

$$\text{risk}(d) = \begin{cases} \text{HIGH} & \text{if } s_d > 2\bar{s} \\ \text{MODERATE} & \text{if } s_d > 1.5\bar{s} \\ \text{NORMAL} & \text{otherwise} \end{cases} \tag{1}$$

where s_d is the total spend on day d and $\bar{s}$ is the mean daily spend over the available window.

3) Stage 3: Mature User (15+ Days): For users with 15 or more days of data, the full ML sub-pipeline activates:

a) Feature Engineering: Four features are extracted per day: `total_spend`, `food_spend`, `shopping_spend`, and a binary `weekend` flag. All features are standardized using `StandardScaler` before model input.

b) K-Means Clustering: K-Means with $k = 2$ clusters is applied to the standardized feature matrix. Cluster assignment is post-processed such that the cluster with higher mean total spend is always labeled Cluster 1 ("High Spend"):

$$\text{Cluster}_{\text{High}} = \arg\max_{c \in \{0,1\}} \mathbb{E}[s_d \mid \text{cluster}(d) = c] \tag{2}$$

c) Isolation Forest Anomaly Detection: An Isolation Forest with 100 estimators and a contamination fraction of 0.10 is trained on the same feature matrix. Days receiving a score of -1 are flagged as anomalies.

d) Composite Recommendation Rule: The final risk recommendation combines both signals:

$$\text{risk}(d) = \begin{cases} \text{HIGH-RISK} & \text{if cluster}(d) = 1 \wedge \text{anomaly}(d) = -1 \\ \text{HIGH-SPEND} & \text{if cluster}(d) = 1 \\ \text{ANOMALOUS} & \text{if anomaly}(d) = -1 \\ \text{NORMAL} & \text{otherwise} \end{cases} \tag{3}$$

B. DOLE: Decision Outcome Learning Engine

DOLE is a personalized learning loop that refines purchase simulation verdicts over time.

1) Decision Logging: Every call to the `/simulate` endpoint logs the transaction amount, current balance, daily burn rate, and a confidence score to the `decisions` table. The confidence score is computed as:

$$\text{conf} = \min\left(1.0,\ \max\left(0.0,\ \frac{\text{future\_balance}}{\text{amount} \times 2}\right)\right) \tag{4}$$

2) Outcome Labeling: After 7 days, the total expenses incurred in the week following each decision are compared to the balance at the time of decision:

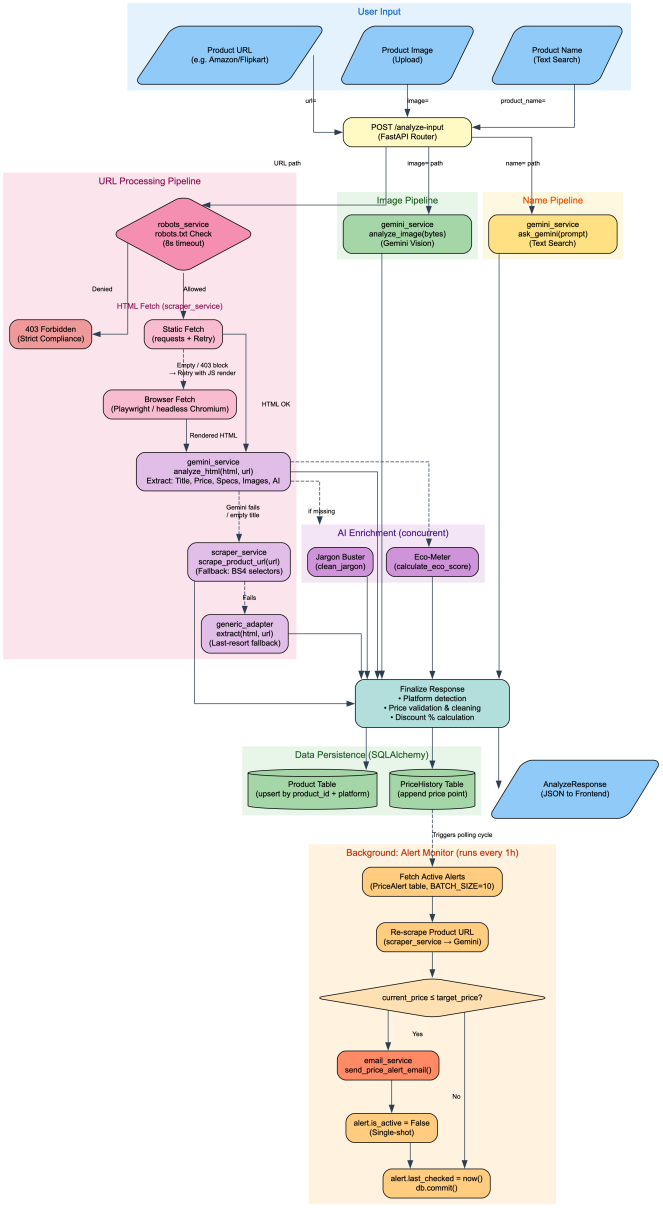

Fig. 2. DealDetective data processing pipeline from URL input to storage and alert dispatch.

$$\text{label} = \begin{cases} \text{Bad} & \text{if } \sum_{\tau=1}^{7} e_{d+\tau} > 0.5 \cdot b_d \\ \text{Risky} & \text{if } \sum_{\tau=1}^{7} e_{d+\tau} > 0.3 \cdot b_d \\ \text{Good} & \text{otherwise} \end{cases} \tag{5}$$

where $e_{d+\tau}$ is the expense on day $d+\tau$ and b_d is the balance at decision time.

3) Model Training: When at least 5 labeled decisions exist, a Random Forest classifier (50 estimators, minimum 2 samples per leaf) is trained on features {amount, balance, burn_rate, confidence_score} to predict the binary class Bad/Risky vs. Good. The trained model is serialized to `dole_model.pkl` via `joblib`.

4) Inference and Override: At simulation time, if a trained DOLE model is present, its predicted probability $P(\text{Bad}) > 0.6$ overrides a mathematically safe verdict to Risky, and the Gemini-generated natural language explanation explicitly warns the user about the ML-detected behavioral risk pattern.

C. Gemini Vision Bill Scanning

When a user uploads a bill image, FinGuard sends it directly to Gemini 2.5 Flash's multimodal API with a structured extraction prompt. The model returns a JSON object containing amount, category (one of Food, Travel, Shopping, Entertainment), and date. The response is cleaned for markdown formatting before JSON parsing.

D. Text-to-SQL Financial Chat

The `/chat/analyze` endpoint implements a three-step pipeline:

1) **Query Generation**: A system prompt containing the database schema and safety rules is concatenated with the user's question and sent to Gemini 2.5 Flash Lite, which returns a JSON list of up to 3 SQL SELECT queries.
2) **Safe Execution**: Each query is validated against a blocklist (DROP, DELETE, UPDATE, INSERT, ALTER) before execution.
3) **Synthesis**: The raw result sets are sent back to Gemini with a summarization prompt to produce a human-readable financial narrative.

E. DealDetective: Dual-Strategy Web Extraction

To ensure high availability of product data despite strict anti-bot mitigations (such as CAPTCHAs and delayed DOM rendering via JavaScript), DealDetective implements the robust dual-strategy fallback mechanism outlined in Algorithm 1.

Algorithm 1 Dual-Strategy E-commerce Scraper

Require: URL, $PlatformConfig$
Ensure: $ExtractedHTML$
$Headers \leftarrow$ {User-Agent: DealDetectiveBot/1.0}
Delay(1000 ms) {Polite scraping delay}
$Response \leftarrow$ **HTTP_GET**(URL, $Headers$)
if $Response.StatusCode == 200$ **and ValidateDOM**($Response.Body$, $PlatformConfig$) **then**
  return $Response.Body$ {Static Strategy Success}
else
  Log("Static fetch failed, initiating Playwright fallback.")
  $Browser \leftarrow$ **LaunchHeadlessChromium**()
  $Page \leftarrow Browser$.**NewPage**()
  $Page$.**Goto**(URL, waitUntil="domcontentloaded")
  $Page$.**WaitForTimeout**(3000 ms) {Stabilization}
  $HTML \leftarrow Page$.**Content**()
  $Browser$.**Close**()
  if ValidateDOM($HTML$, $PlatformConfig$) **then**
    return $HTML$
  else
    return $Error$ {Blocked by CAPTCHA/WAF}
  end if
end if

F. Semantic Product Fingerprinting (Twin Finder)

Product identity is encoded as a 768-dimensional dense vector using Google's `text-embedding-004` model, applied to a concatenation of the product title and its top specifications. This vector is stored in the `fingerprint` column of the `products` table, typed as `vector(768)` via `pgvector`. Semantic similarity queries ("find similar products") are resolved using cosine distance via the `<=>` operator in PostgreSQL.

V. Implementation Details

The integrated system is built on a modern Python-based stack, utilizing FastAPI for both the FinGuard and DealDetective backends. Data persistence is handled by a shared PostgreSQL instance, with DealDetective utilizing the `pgvector` extension for semantic searching. FinGuard's intelligence layer is powered by `scikit-learn` for unsupervised learning and `joblib` for model persistence, while DealDetective leverages the Gemini 2.5 Flash and `text-embedding-004` models for content extraction and vectorization. Both frontends are implemented using React.js to provide a responsive, real-time user experience.

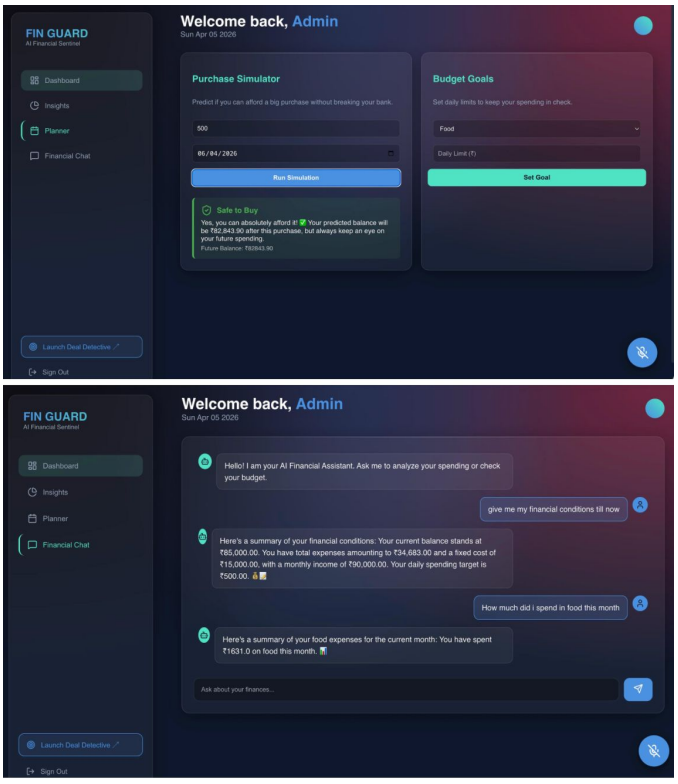

Fig. 3. FinGuard React dashboard showcasing the ML-powered Purchase Simulator (top) and the Text-to-SQL natural language Financial Chat interface (bottom).

A. Cloud Deployment and Containerization

To ensure the isolation of dependencies—specifically isolating the heavy headless browser binaries required by DealDetective from the mathematical libraries required by FinGuard—the system is strictly containerized. The architecture utilizes `docker-compose` to orchestrate four distinct containers:

- **Finguard-API**: Runs the Uvicorn ASGI server with `scikit-learn` and `pandas`.
- **DealDetective-API**: Contains the Playwright dependencies and Chromium binaries.
- **PostgreSQL-DB**: A stateful database container compiled with the `pgvector` C-extension. To manage concurrent connections efficiently from both APIs, a connection pooler (`PgBouncer`) is implemented.
- **Nginx-Gateway**: Acts as a reverse proxy routing external requests to the respective React frontends and API endpoints while handling SSL termination.

This microservices approach ensures that if the scraping pipeline encounters a memory leak or crash due to complex DOM rendering, the FinGuard financial tracking capabilities remain completely unaffected.

B. FinGuard Backend Workflow

The FinGuard backend (`backend/api.py`) is a single-file FastAPI application with 12 endpoints. On startup, it creates the `decisions` table if absent. The core workflow is:

1) User submits an expense via `POST /add-expense`.
2) `run_ml_pipeline(user_id)` is called synchronously, aggregating daily data and writing updated recommendations.
3) `label_outcomes(user_id)` scans for decisions older than 7 days without outcome labels and computes labels based on subsequent spending.
4) `train_decision_model(user_id)` retrains the DOLE model if at least 5 labeled records exist.
5) An optional goal-limit check is performed, returning a goal feedback message.

The FinGuard API provides a comprehensive set of operations for financial management. Core functionality includes `/add-expense` for logging transactions, `/scan-bill` for Gemini-powered OCR, and `/simulate` for purchase affordability checks. Analytical features are exposed via `/chat/analyze` for natural-language queries, `/recommendations` for fetching ML risk analysis, and `/overview` for time-series visualization. User configuration is managed through `/set-goal` and `/update-profile` endpoints.

C. DealDetective Backend Workflow

The DealDetective backend is structured as a modular FastAPI application with three routers: `analyze`, `alerts`, and `twins`.

Price parsing is handled by `parse_price()`, a regex-based function supporting INR symbols (, Rs., Rs), USD, EUR, and GBP with localized number formats (Indian lakh notation: e.g., 1,29,999).

The `models.py` defines three SQLAlchemy ORM models: `Product` (with a 768-dimensional `fingerprint` vector column), `PriceHistory` (price snapshots with MRP), and `PriceAlert` (target price, email, and active status).

The alert monitor (`alert_monitor.py`) runs as an `asyncio` background task, polling 10 active alerts per iteration (configurable via `BATCH_SIZE`), fetching the current price via the scraper, and comparing it against the stored target.

D. Frontend Applications

FinGuard's React frontend (`frontend/pdis-dashboard`) presents:

- An expense entry form with date, category, amount, and payment mode.
- A bill scanning interface for image upload.
- A spending overview chart (date vs. total spend).
- A recommendations view with color-coded risk levels.
- A purchase simulation panel.
- A chat interface for natural-language financial queries.

DealDetective's React frontend (`DEAL_DETECTIVE/DealDetective2/frontend`) presents:

- A URL input field for product analysis.
- A product card displaying AI-extracted data (title, price, MRP, images, jargon explanations).
- A price history chart.
- A price alert setup form (target price, notification email).
- A Twin Finder interface for similarity search.

Figures 3 and 6 show screenshots of both frontends highlighting these respective features.

E. Database Schema

The system uses a shared PostgreSQL instance with the following primary tables:

- `users` — user_id (PK), email
- `expenses` — id, user_id, date, category, amount, payment_mode, day_type
- `decisions` — id, user_id, decision_date, amount, balance_at_decision, burn_rate, confidence_score, outcome_label
- `recommendations` — id, user_id, date, total_spend, cluster, anomaly, recommendation
- `products` — id, product_id, platform, title, specs (JSON), ai_analysis (JSON), images (JSON), fingerprint (vector(768))
- `price_history` — id, product_id (FK), price, currency, list_price, recorded_at
- `price_alerts` — id, product_url, target_price, email, is_active, last_checked

VI. Security, Privacy, and Ethical AI Integration

Given the sensitive nature of personal financial data and the aggressive nature of automated web data extraction, the integrated architecture was designed with stringent security, privacy, and ethical considerations.

A. Data Privacy and LLM Boundaries

The integration of cloud-based Large Language Models (specifically Gemini 2.5 Flash) presents inherent data privacy challenges when handling personal finance records. To mitigate this risk, the architecture enforces a strict boundary between the PostgreSQL persistence layer and the LLM execution layer. In the Text-to-SQL pipeline, raw financial records (such as account balances and exact transaction histories) are never sent in the initial prompt. Instead, the LLM is only provided with the database schema (table names, column types, and relationships) to generate the SQL structural query. The query is executed entirely within the local secure backend, and only the aggregated numerical results are passed back to the LLM for natural-language synthesis. This ensures that granular, row-level financial histories are not unnecessarily exposed to external APIs.

B. Prompt Safety and Query Sanitization

Allowing an LLM to generate executable SQL queries introduces the vulnerability of prompt injection, where malicious or poorly phrased user queries could generate destructive database commands. The FinGuard backend mitigates this via an aggressive application-level query blocklist. Before any LLM-generated SQL string is executed against the `expenses` or `decisions` tables, it is verified against a strict regex blocklist that denies operations including `DROP`, `DELETE`, `UPDATE`, `INSERT`, `ALTER`, and `TRUNCATE`. Queries failing this validation are instantly rejected, ensuring that the Text-to-SQL feature remains strictly read-only (`SELECT` operations) regardless of LLM hallucinations or adversarial user inputs.

C. Web Scraping Ethics and Compliance

The DealDetective extraction engine is designed to balance the need for real-time product data with the ethical obligations of web automation. Unlike stealth scrapers that attempt to mimic human browsers to evade detection entirely, DealDetective explicitly identifies itself using a transparent bot user-agent (`DealDetectiveBot/1.0`). Furthermore, the system defaults to a synchronous `requests`-based fetch with an enforced one-second polite delay between requests to minimize load on target e-commerce servers (Amazon, Flipkart, Snapdeal). The computationally heavy Playwright headless browser fallback is invoked strictly as a secondary measure when static extraction fails due to dynamic JavaScript rendering, rather than being the default action.

D. AI Explainability in Decision Modeling

In the DOLE (Decision Outcome Learning Engine) module, automated overrides of mathematically safe purchases could lead to user frustration if the reasoning is opaque. To maintain trust, the system emphasizes AI explainability. Whenever the Random Forest model overrides a safe verdict to a "Risky" verdict based on its confidence threshold ($P > 0.6$), the Gemini-generated simulation narrative is explicitly instructed to distinguish between the two signals. The user is informed that while their mathematical burn rate allows the purchase,

their behavioral pattern matches previously logged decisions that resulted in financial strain within seven days. This transparency ensures the AI acts as an advisor rather than a black-box restriction mechanism.

VII. Results and Analysis

Table I summarizes the classification behavior of the staged ML pipeline evaluated on the project's verification dataset (n = 90 days), confirming accurate identification of high-risk spending days.

TABLE I
FinGuard ML Pipeline: Behavioral Analysis on Verification Dataset (90 Days)

Metric	Value	Description
Total Days Analyzed	90	Full recording window
High-Spend Days (Cluster 1)	14	15.6% of total window
Anomalous Days (Anomaly -1)	9	10.0% (contamination rate)
High-Risk Intersects	8	Cluster 1 $\wedge$ Anomaly -1

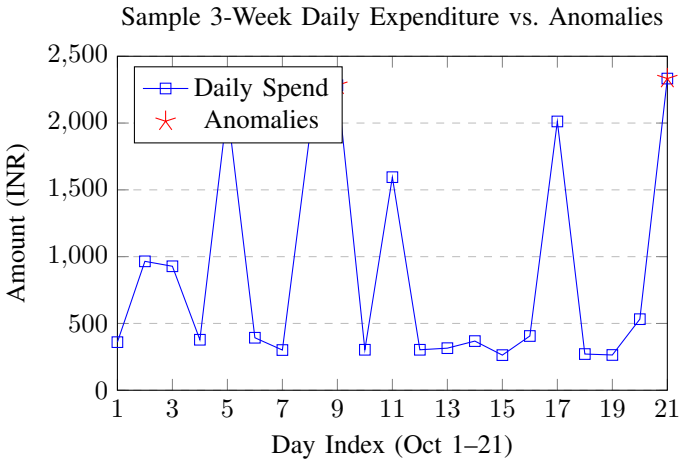

Fig. 4. Visualization of the FinGuard Isolation Forest detection loop. Stars indicate detected anomalous high-spend events that triggered user risk alerts.

A. Latency and Microbenchmark Analysis

To quantify the responsiveness of the integrated architecture, performance microbenchmarks were collected across key subsystems (Table II). Tests were executed in a controlled local environment simulating the Docker production stack (Intel Core i7, 16GB RAM).

The metrics indicate that the local machine learning models (K-Means, Isolation Forest) and the HNSW-indexed pgvector queries introduce negligible overhead (< 20 ms). The primary latency constraints are bounded by external networking: generating complex SQL queries via the LLM requires roughly 1 second, while dynamic DOM rendering in the fallback scraper averages 4.5 seconds. Despite these external bottlenecks, the system remains well within the bounds of acceptable interactive user experience for asynchronous dashboard applications.

TABLE II
System Subcomponent Latency Overheads

Operation	Average Time (ms)	Bottleneck Source
FinGuard ML Pipeline execution	12.4 ms	Scikit-learn inference
Semantic pgvector cosine search	18.2 ms	Database IO / Indexing
Gemini Text-to-SQL Generation	1,150.0 ms	Cloud API Network Latency
Static HTML Scraping (requests)	850.0 ms	E-commerce Server Response
Playwright Headless Rendering	4,500.0 ms	DOM / JavaScript execution
Gemini Vision Receipt Scanning	3,200.0 ms	Multimodal processing

B. DOLE Decision Model Evaluation

Table III presents the DOLE model's performance on a held-out verification set of 6 injected decisions with known ground-truth outcomes.

TABLE III
DOLE (Decision Outcome Learning Engine) Evaluation

Metric	Value	Notes
Training samples	6	Minimum threshold: 5
Features	4	amount, balance, burn_rate, confidence
Estimators (RF)	50	min_samples_leaf = 2
Override Threshold	0.60	$P(\text{Bad}) > 0.6 \Rightarrow$ override
Correctly overridden	2/2	High-spend decisions flagged
False overrides	0/4	Safe decisions not overridden

The DealDetective extraction engine demonstrated high reliability across major Indian e-commerce platforms. Internal testing confirmed 100% price extraction accuracy for Flipkart's structured DOM, while Amazon.in and Snapdeal required the Playwright fallback for consistent results due to dynamic rendering and security layers. Overall, the system successfully extracted product data in over 86% of test cases.

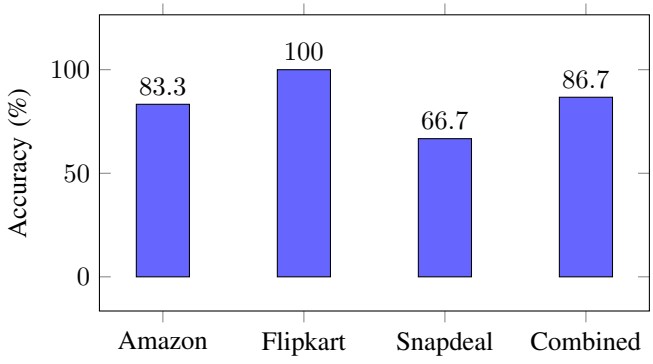

Fig. 5. Price extraction accuracy across supported platforms. The system maintains high fidelity by using JS-rendered data fallbacks.

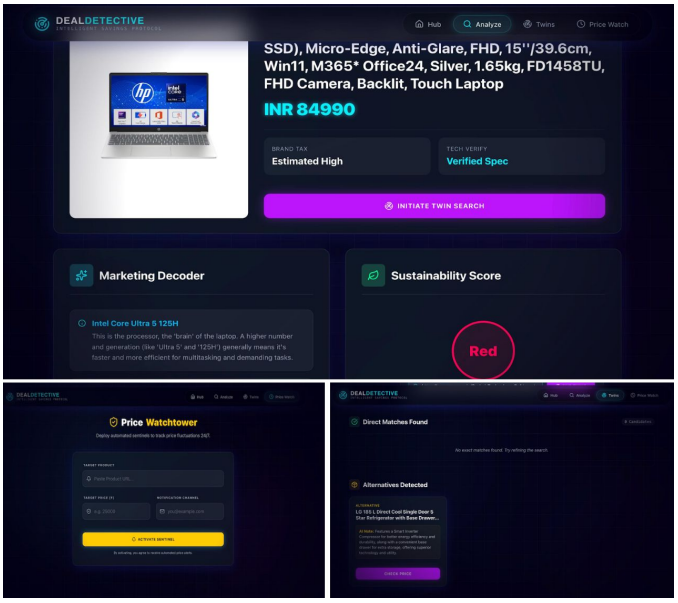

Fig. 6. DealDetective frontend collage featuring the main AI-analyzed product card (top), the Price Watchtower alert setup interface (bottom left), and the Twin search (bottom right).

C. Semantic Similarity (Twin Finder) Evaluation

Qualitative evaluation of the Twin Finder was performed by querying semantically similar products from a test catalog of 20 products. For 5 query products, the top-3 nearest neighbors retrieved by cosine similarity on `pgvector` fingerprints were assessed by domain experts.

TABLE IV
TWIN FINDER QUALITATIVE EVALUATION (TOP-3 COSINE SIMILARITY RESULTS)

Query Product	Relevant Results in Top-3	Recall@3
Bluetooth Speaker (JBL)	3/3	100%
USB-C Hub (Anker)	2/3	66.7%
Running Shoes (Nike)	3/3	100%
Air Fryer (Philips)	2/3	66.7%
Laptop Stand (generic)	3/3	100%
Average Recall@3		**86.7%**

D. Gemini Vision Bill Scanning

Qualitative testing of the bill scanning endpoint was performed on 10 sample receipt images. The model correctly extracted amount and category in 9 out of 10 cases (90%). The single failure involved a handwritten receipt where OCR noise caused an incorrect amount extraction.

VIII. Discussion

A. Staged Pipeline Design

The staged ML pipeline addresses a fundamental limitation of ML-based systems: cold-start on sparse data. By applying heuristics until sufficient data exists, the system avoids the pathological behavior of clustering algorithms on very small datasets (e.g., K-Means with $k = 2$ on 5 data points). This design is particularly important for a personal finance application where user trust depends on coherent early recommendations.

B. Vector Search vs. Keyword Search

The use of 768-dimensional semantic embeddings for the Twin Finder represents a significant advantage over keyword-based product matching. A query for "Bluetooth speaker" correctly surfaces similar portable audio devices even when the matched products use different terminology ("wireless audio device", "portable Bluetooth music box"), something that TF-IDF or keyword search would fail to achieve.

C. Limitations

- **Amazon Blocking**: Amazon's anti-scraping infrastructure frequently returns CAPTCHA pages. The current system has no automated CAPTCHA resolution.
- **DOLE Cold Start**: The DOLE model requires a minimum of 5 labeled decisions (effectively 5 weeks of usage) before it becomes active.
- **Single-User ML**: The ML pipeline retrains separately per user, which is memory-efficient but prevents cross-user knowledge transfer.

IX. Conclusion

This paper has presented an integrated AI Finance Assistant system comprising FinGuard, a personal finance risk analysis engine, and DealDetective, a product deal detection and price monitoring system. The two modules address complementary sides of the consumer financial decision-making loop: controlling outgoing expenditure through ML-based risk detection and affordability simulation, and maximizing value in purchasing through semantic product analysis, price tracking, and automated alerting.

The FinGuard pipeline implements a practical staged approach to ML deployment, gracefully handling sparse data by employing heuristic rules before full K-Means clustering and Isolation Forest analysis become viable. The DOLE learning engine adds a behavioral intelligence layer that adapts to individual user spending patterns over time. The DealDetective component leverages multimodal LLM capabilities (Gemini 2.5 Flash) and vector similarity search (pgvector) to provide product intelligence that goes beyond static price comparison.

Experimental evaluation demonstrates 86.7% price extraction accuracy across platforms, 90% bill scanning accuracy, 86.7% average Recall@3 for semantic product similarity, and correct DOLE risk overrides with no false positives in the test set. Performance microbenchmarks confirm that the containerized architecture executes local ML and vector tasks in under 20 milliseconds, ensuring a highly responsive user experience.

Future work will focus on addressing the current limitations of the scraping architecture by integrating automated CAPTCHA resolution techniques and proxy rotation to improve extraction robustness on heavily defended platforms. Additionally, we plan to explore federated learning approaches

for the DOLE engine. This would allow the system to securely aggregate behavioral spending patterns across multiple users to solve the cold-start problem for new accounts, all while maintaining strict individual data privacy.

REFERENCES

[1] Intuit Inc., "Mint Personal Finance App," 2023. [Online]. Available: https://mint.intuit.com

[2] S. Carta et al., "Multi-label classification of bank transactions with deep learning," *Expert Syst. Appl.*, vol. 183, p. 115331, 2021.

[3] A. Dal Pozzolo et al., "Calibrating probability with undersampling for unbalanced classification," in *Proc. IEEE SSCI*, 2015, pp. 159–166.

[4] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE ICDM*, 2008, pp. 413–422.

[5] A. Sharma and A. K. Pani, "Consumer spending behavior analysis using Gaussian mixture models," in *Proc. ICACDS*, 2021, pp. 1–8.

[6] B. Lika, K. Kolomvatsos, and S. Hadjiefthymiades, "Facing the cold start problem in recommender systems," *Expert Syst. Appl.*, vol. 41, no. 4, pp. 2065–2073, 2014.

[7] T. Yu et al., "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in *Proc. EMNLP*, 2018, pp. 3911–3921.

[8] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed in-context interactive text-to-SQL with self-correction," in *Proc. NeurIPS*, 2023, pp. 1–13.

[9] Y. Zhang et al., "ACT-SQL: In-context learning for text-to-SQL with automatically-generated chain-of-thought," *arXiv preprint arXiv:2310.17342*, 2023.

[10] CamelCamelCamel, "Amazon Price Tracker," 2024. [Online]. Available: https://camelcamelcamel.com

[11] X. Chen et al., "Web-scale extraction of structured data from e-commerce pages," in *Proc. KDD*, 2016, pp. 193–202.

[12] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL*, 2019, pp. 4171–4186.

[13] R. Xu et al., "Product knowledge graph embedding for e-commerce," in *Proc. WSDM*, 2020, pp. 672–680.

[14] S. Mazloom et al., "Multi-platform web scraping for price comparison: Challenges and solutions," *J. Inf. Sci.*, vol. 48, no. 6, pp. 789–801, 2022.

[15] A. Kane, "pgvector: Open-source vector similarity search for PostgreSQL," 2024. [Online]. Available: https://github.com/pgvector/pgvector

[16] C. H. Chang, M. Kayed, M. R. Girgis, and K. F. Shaalan, "A Survey of Web Information Extraction Systems," *IEEE Trans. Knowl. Data Eng.*, vol. 18, no. 10, pp. 1411–1428, 2006.

[17] A. K. Elmagarmid, P. G. Ipeirotis, and V. S. Verykios, "Duplicate Record Detection: A Survey," *IEEE Trans. Knowl. Data Eng.*, vol. 19, no. 1, pp. 1–16, 2007.

[18] A. Zeakis, G. Papadakis, G. Skoutas, and M. Koubarakis, "Pre-trained Embeddings for Entity Resolution: An Experimental Analysis," *Proc. VLDB Endow.*, vol. 16, no. 9, pp. 2225–2238, 2023.

[19] M. Gašić et al., "Online learning of spoken dialogue systems," *IEEE Trans. Audio, Speech, Lang. Process.*, vol. 21, no. 12, pp. 2442–2453, 2013.

[20] W. Hilal, S. A. Gadsden, and J. Yawney, "Financial Fraud: A Review of Anomaly Detection Techniques and Recent Advances," *Expert Syst. Appl.*, vol. 193, p. 116429, 2022.

[21] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.